

Sales Visit Planner - Stage 2 Development

Name: Luke Doyle

Student ID: D00255656

1. Introduction
 - 1.1 Recap on Architecture
2. Kubernetes Deployment
 - 2.1 Cluster Setup
 - 2.2 Stateless Workloads
 - 2.3 Stateful Workloads
 - 2.4 Ingress Gateway
 - 2.5 Helm Deployment
3. Istio Architecture Overview
 - 3.1 Sidecar VS Ambient Istio
4. Security
 - 4.1 Mutual TLS
 - 4.2 Zero-Trust Authorisation
 - 4.3 Defence in Depth: Calico and Istio
5. Traffic Control
 - 5.1 Load Balancing
 - 5.2 Canary Deployment
 - 5.3 Circuit Breaker
 - 5.4 Rate Limiting
6. Observability
 - 6.1 Metrics - Prometheus
 - 6.2 Distributed Tracing - Jaeger
 - 6.3 Topology Visualisation - Kiali
7. Testing
 - 7.1 Functional Tests
 - 7.2 Stress Tests
 - 7.3 Fault Injection
 - 7.4 Pod Failure
 - 7.5 Rate Limiting
8. Reflection
9. Use of AI

1. Introduction

The Sales Visit Planner (SVP) is a microservice application made for field sales representatives to plan and manage their workday.

The system consists of four FastAPI services: the customer and user service, planning service,

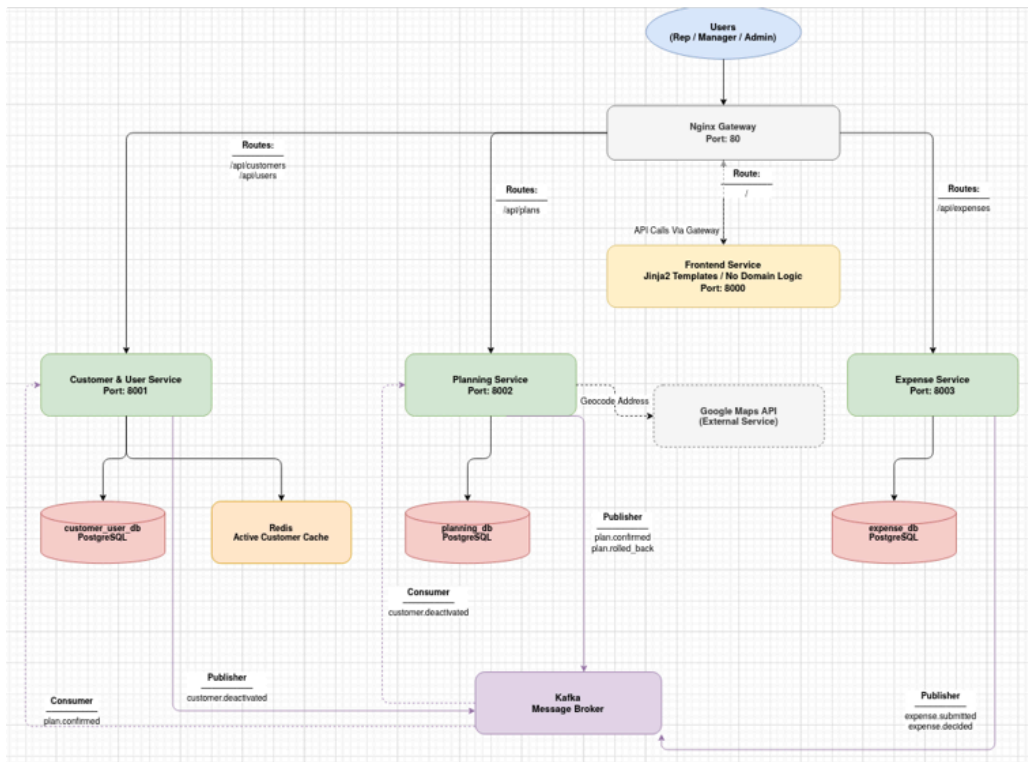
expense service, and frontend service. All services have a distinct PostgreSQL database, any cross-service coordination occurs on Kafka.

The application with the microservice architecture was the target of stage 1, focusing on the Saga, Transactional Outbox and CQRS patterns.

This report covers stage 2, which deploys the system to a three-node KinD Kubernetes cluster with Calico as the CNI and Istio 1.21.0 as the service mesh, addressing security, traffic control, observability, and resilience.

```
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ kubectl get pods -n svp
NAME                                READY   STATUS    RESTARTS   AGE
customer-user-db-0                  1/1     Running   0           13m
expense-db-0                        1/1     Running   0           13m
kafka-0                            1/1     Running   0           13m
planning-db-0                      1/1     Running   0           13m
redis-0                            1/1     Running   0           13m
svp-customer-user-service-bc596469-ktnlx 2/2     Running   0           12m
svp-expense-service-6d75dc4987-x4dl2    2/2     Running   0           12m
svp-frontend-service-6459b865ff-tt8k7    2/2     Running   0           12m
svp-planning-service-6d76f55fbf-ktp7x    2/2     Running   0           12m
zookeeper-0                        1/1     Running   0           13m
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ kubectl get peerauthentication -n svp
NAME      MODE    AGE
default   STRICT  12m
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ kubectl get authorizationpolicy -n svp
NAME                                AGE
allow-customer-user-service        12m
allow-expense-service              12m
allow-frontend-service             12m
allow-planning-service             12m
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ kubectl get virtualservice -n svp
NAME                                GATEWAYS    HOSTS    AGE
customer-user-service-vs           ["svp-gateway"]  ["*"]    12m
expense-service-vs                  ["svp-gateway"]  ["*"]    12m
frontend-service-vs                 ["svp-gateway"]  ["*"]    12m
planning-service-vs                 ["svp-gateway"]  ["*"]    12m
• (.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ kubectl get destinationrule -n svp
NAME                                HOST    AGE
customer-user-service-dr            customer-user-service  12m
expense-service-dr                  expense-service        12m
frontend-service-dr                 frontend-service       12m
planning-service-dr                 planning-service       12m
○ (.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$
```

1.1 Recap on Architecture



The system is composed of 4 application services, an Nginx Gateway, Apache Kafka, along with both Redis, and PostgreSQL databases. All components run as Docker containers.

An Nginx Gateway acts as a single point of entry for all external traffic on port 80. It strips the `/api` prefix from incoming requests and routes by path to the appropriate backend service(s). Nothing is reachable directly.

Each service has its own PostgreSQL database. No service can query another service's database directly for either read or write. This helps keep the system secure and scalable. If a service needs data from another service's database, it either queries the API through the gateway or picks up a Kafka event. This keeps the services loosely coupled and allows each service's schema to evolve independently, following the core idea of Microservice architecture.

The four services are as follows.

- The Customer and User Service handles customer records, user accounts, and the active customer list. The list gets cached in Redis, this reduces database queries. This service is also where customer deactivation happens, it publishes a ``customer.deactivated`` event to Kafka when a customer is deactivated.
- The Planning Service owns the full “day plan” lifecycle. It runs saga orchestration, manages the transactional outbox, calls the Google Maps API, and maintains CQRS read projections. Confirming a plan writes the status change and the outbox event in one transaction.

- The Expense Service handles submission, approval, and rejection of expense claims. It publishes Kafka events when decisions are made and keeps a CQRS read model that managers query for expense history.
- The Frontend Service just renders Jinja2 templates. No domain logic, no database access. Everything it displays comes from the backend services via Nginx.

Kafka connects the Customer and User Service and the Planning Service. The two never call each other directly. All coordination goes through ``plan.confirmed``, ``plan.rolled_back``, and ``customer.deactivated`` topics.

NOTE: No Major Changes have been made to the Application Architecture

2. Kubernetes Deployment

The system is deployed to a three-node KinD cluster running one control-plane node and two worker nodes. The cluster runs Calico as the CNI for enforcing network policy at Layer 3 and 4, with Istio sidecars for Layer 4 and Layer 7 application protocol. Think of Calico as “Who can talk to who” and Istio as “What can be said”.

All services run in the `svp` namespace. The complete deployment is done through a Helm chart.

It is then brought up using a set of Python deployment scripts. These scripts sequentially handle cluster creation, image building, image loading to KinD, and chart installation.

2.1 Cluster Setup

KinD cluster is defined in `kind-config.yaml` with three nodes: one control-plane and two workers. The control-plane node exposes `hostPort 80`, where the external traffic hits the Istio ingress gateway. After the cluster creation process, we apply Calico CNI manifests.

The scripts in `/plat_scripts` wait for all nodes to reach a ready state.

Their run order are;

```
1 python3 plat_scripts/deploy.py
2 python3 plat_scripts/install_istio.py
3 python3 plat_scripts/install_observability.py
```

As part of the Helm chart installation, `svp` namespace is created and labelled “istio-injection=enabled”.

This tells Istio to automatically inject an Envoy sidecar into every pod which starts in that namespace. The Istio reference does not need to be included in the application Dockerfile or deployment manifest as the injection happens during pod creation.

Each application pod has the annotation `sidecar.istio.io/excludeOutboundPorts: "5432,6379,9092,2181"`, which excludes PostgreSQL, Redis, Kafka, and Zookeeper traffic from the Envoy interception.

Mesh-level mTLS is not required on these internal data-tier connections, and not enabling it means there do not have to be `ServiceEntry` resources for every host. This simplifies the mesh.

2.2 Stateless Workloads

A stateless workload holds no data locally. Since any pod can handle any request allowed by Istio, they are de-coupled and independent. If a pod is killed and replaced nothing is lost. All states exist in a database, cache, or message broker.

The four services,; customer-user, planning, expense, and frontend, all function as stateless workloads in deployment.

All persistent data lives in the databases, and the Redis cache is treated as expendable.

A `Deployment` for each service in Kubernetes is set up, where each service uses its own container image built from the project root. The image is loaded directly into KinD and not pulled from a registry, as such, the setup will remain self-contained for the assessment.

2.3 Stateful Workloads

A stateful workload stores information that needs to remain even if a pod is deleted. Each pod, unlike stateless ones, comes with a stable identity and possesses its own unique storage. When a pod is restarted, it will still connect to the same `PersistentVolumeClaim` instead of starting again with no memory of what came before.

The components of the data tier hold state, which needs to survive pod restarts, unlike the application services. Rather than `Deployments`, these are put into action as `StatefulSets`, assigning a stable network identity and its own `PersistentVolumeClaim` to each pod.

The namespace called `svp` has three Postgres instances which are stateful workloads. One instance is owned by each of the three services: `svp-customer-user-db`, `svp-planning-db`, `svp-expense-db`. Besides this are Redis which is used by the customer-user service only for the active cache of customers.

Finally, there is Kafka with its Zookeeper coordinator. Each PostgreSQL instance is dedicated to its own service. No service may directly query another service's database.

Cross-service data access defaults to Kafka events, or in rare event the API itself.

2.4 Ingress Gateway

The ingress gateway for Istio listens on port 80 for external traffic and is port-forwarded to localhost:8080 for local access.

A single `Gateway` resource selects all hosts, and four `VirtualService` resources handle URI prefix routing. `/api/users` and `/api/customers` go to user-service `/api/plans` goes to planning service, `/api/expenses` goes to expense service and everything else is caught in the frontend service.

All traffic must pass through the gateway; no device on the cluster accepts connections directly.

This means the ingress gateway acts as a single point of entry into the system. In addition, this is where Istio applies its routing rules, traffic policies, and telemetry collection, before the request reaches any service pod.

2.5 Helm Deployment

The entire application is packaged as a Helm chart located in `helm/svp/`. All four application services, databases, Redis, Kafka, Zookeeper, and the related Kubernetes resources `ConfigMaps`, `Services`, and `PersistentVolumeClaims` are included. Deploying the complete stack is a simple helm install command, which makes it simple to bring the application up in a clean cluster for evaluation.

The deployment is managed by three Python scripts.

- `deploy.py` creates the KinD cluster, applies Calico, builds and loads the Docker images, and installs the Helm chart.
- `install_istio.py` installs Istio with the demo profile, labels the `svcp` namespace for sidecar injection, restarts all deployments to inject sidecars, and then applies the security policies, networking resources, and the Telemetry resource.
- `install_observability.py` installs Prometheus, Jaeger, Kiali, and Grafana from the Istio addons directory.

Executing the three scripts in order launches the entire platform from scratch.

3. Istio Architecture Overview

There are two parts to Istio, the control plane and the data plane.

The Istiod binary operates the control plane functionalities. It facilitates the interaction of services with one another, provides service configuration to the proxies, and provides the

Certificate Authority for mTLS.

Each application pod contains Envoy proxy injected as a sidecar, and this is the data plane. Envoy intercepts all incoming and outgoing traffic without the application's knowledge due to transparency. It is here that the enforcement of routing rules, access policies, and telemetry collection occurs. Every Envoy instance receives its configuration from Istiod using the xDS protocol. Thus, routing and policy changes happen at run time, not at deploy time.

3.1 Sidecar VS Ambient Istio

There are two models for deploying Istio; Sidecar and Ambient.

SVP uses the Sidecar model, as an Envoy proxy is injected into every pod. Every application has its own proxy to provide the maximum visibility. It has L4 control and L7 control.

The trade-off for this pattern is the resource overhead, as every pod now carries the sidecar container. Additionally, upgrades require restarting pods to inject the new version of the sidecar into them.

The Ambient model, introduced as beta in Istio 1.21, completely eliminates the per-pod sidecar. It employs a node-specific component named `ztunnel` that manages L4 mTLS and fundamental authorization for every pod on the node.

To enable application-level features like HTTP routing, retries, and fault injection, a `waypoint` proxy is deployed as a sidecar.

This prevents L7 from being necessary for services that don't need it.

The practical distinction is more about the layer you need.

Layer 4 functions at the TCP level; it understands the source, destination, and port but not the contents of the request.

Layer 7 works at the application layer and can view HTTP methods, paths, headers, and gRPC service names.

Most of Istio's useful Traffic Management features require L7.

For SVP, the selection of the sidecar model was appropriate. Since Ambient was still in beta at deployment and the full L7 feature set was needed from day one, mTLS, authorisation policies, fault injection, and distributed tracing all depend on it.

4. Security

The system's security managed at two separate layers.

Calico provides peer-to-peer network policies at Layer 3 and 4 for pod isolation, while Istio offers service-to-service access control at Layer 4 and 7.

Think of Calico as “Who can talk to who” and Istio as “What can be said”.

For any connection to succeed, the request must satisfy both Calico and Istio.

4.1 Mutual TLS

Through a PeerAuthentication resource, mTLS is enforced strictly across `svc` namespace. In strict mode, plaintext connections are completely rejected, every side of a connection must present a valid certificate.

Istiod performs the certificate authority function, automatically issuing and rotating short-lived x.509 certificates for each workload. The Application Code doesn't take part in this.

4.2 Zero-Trust Authorisation

AuthorizationPolicy resources enforce zero-trust authorization. A default deny-all policy for the `svc` namespace blocks all traffic unless permitted explicitly.

Policies specifying ALLOW for each service will functionally open only the paths that are required. The ingress gateway can reach all four services, and the `svc` services can reach each other where required.

Pods that are not in the allowed list cannot connect, even if they are within the same cluster.

4.3 Defence in Depth: Calico and Istio

Both Calico and Istio control access, but do so independently. Calico implements Layer 3 and Layer 4 control by managing which pods establish network connections based on IP and port. The mTLS certificate's workload identity gets used by Istio to enforce at L7. A wrongly configured policy in Istio will not affect Calico, and vice versa.

For any connection to succeed, the request must satisfy both Calico and Istio.

5. Traffic Control

Traffic management in the SVP is controlled by Istio `DestinationRule` and `VirtualService` resources applied to the four services. Load balancing, circuit breaking, and rate limiting are all done at the proxy level, meaning no application code is involved.

5.1 Load Balancing

Each service configures a `DestinationRule` with “LoadBalance-round-robin”. Round-robin evenly distributes requests among available pods in sequence. For stateless services like those

in SVP, it is simple and appropriate as any pod can serve any request, as long as Istio allows the request.

5.2 Canary Deployment

A canary deployment methodically reroutes traffic to a service's updated version pre-commitment for a comprehensive rollout. In Istio a `VirtualService` carries the weighted traffic split to the two subsets in `DestinationRules`. For example, 80% of traffic is routed to v1, 20% to v2. This was not put in place as there is only one instance of each service; however, the internal structure of `DestinationRule` already exists and a weighted `VirtualService` would apply without any required changes to the application.

5.3 Circuit Breaker

Every `DestinationRule` has configurations for outlier detection. If a pod generates consecutive errors within a specific timeframe, it will be temporarily removed from load balancing activities and the residual pods handle the traffic. This stops a single failing pod from getting requests that it cannot handle.

5.4 Rate Limiting

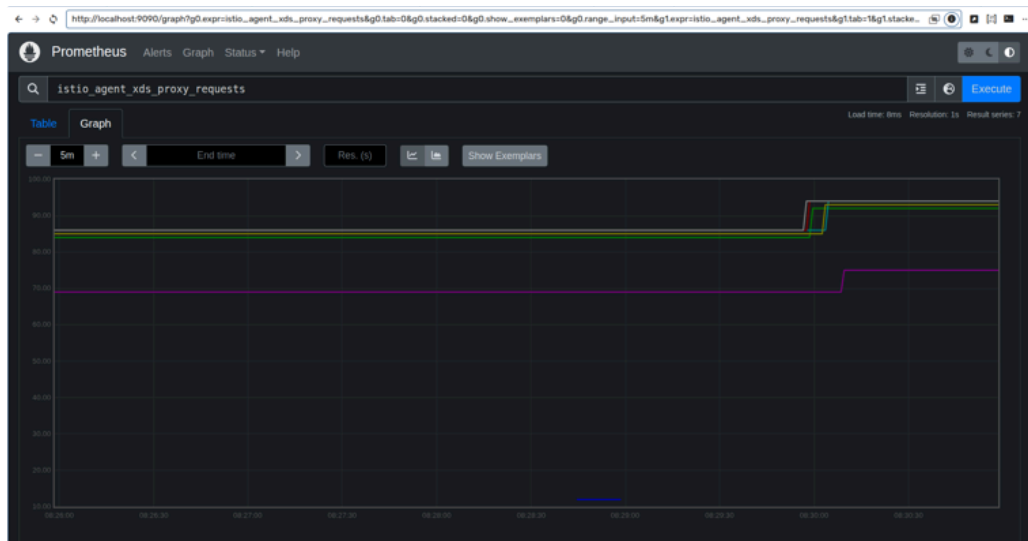
A local token bucket rate limiter is applied to the customer-user service, through an `EnvoyFilter`, set at 10 tokens/sec. Requests that exceed this will receive an HTTP 429 with the `x-local-rate-limit:` header set to true.

This is a limit applicable to each Envoy instance individually, and not a global limit shared by all pods. A true global rate limiter would require an external gRPC Rate Limit Service, which was outside the scope of this project.

6. Observability

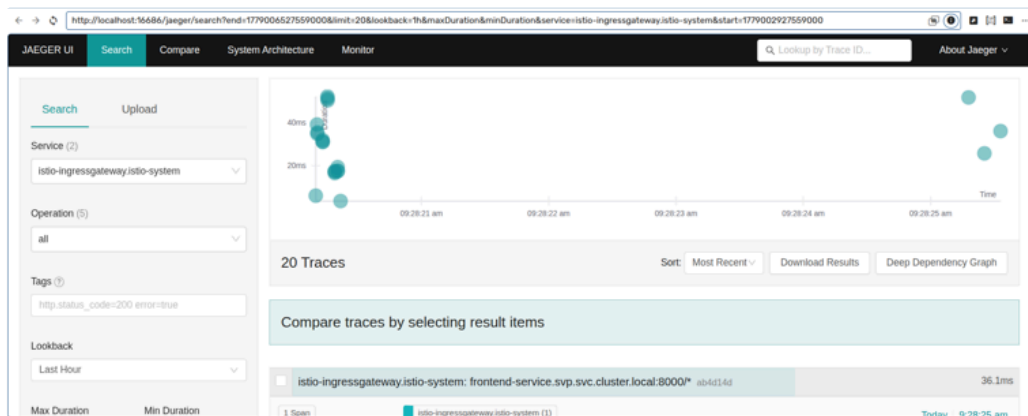
6.1 Metrics - Prometheus

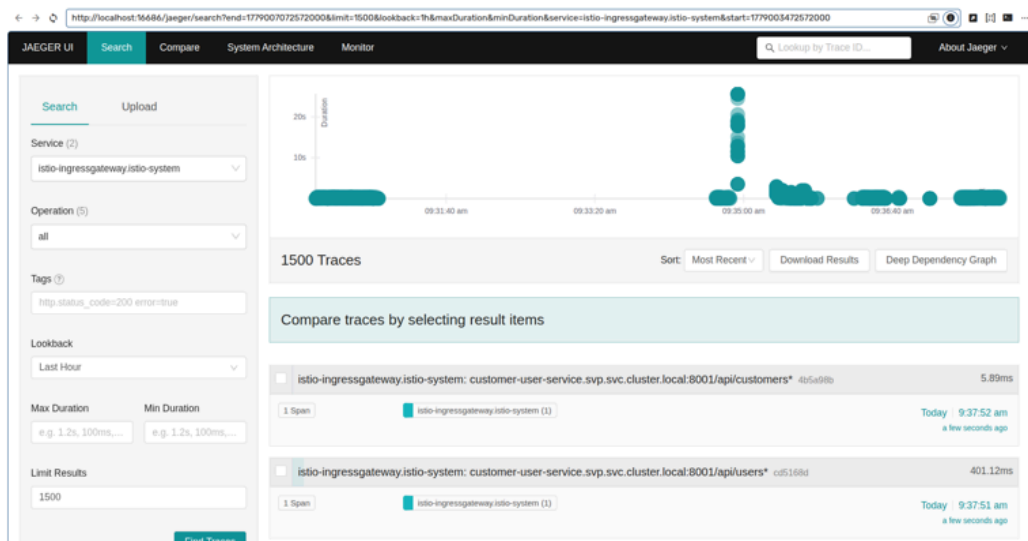
Envoy sidecar in every pod directly scrapes metrics from Prometheus. The individual Envoy agents each use the `istio_agent_xds_proxy_requests` metric to determine an external xDS proxy request. The graph comprises seven streams of data, each corresponding to a pod in the cluster, and a step change is evident roughly around 08:30, when the stress tests began sending traffic. It confirms that Prometheus is receiving telemetry from the mesh layer of all services without requiring instrumentation in the application code.



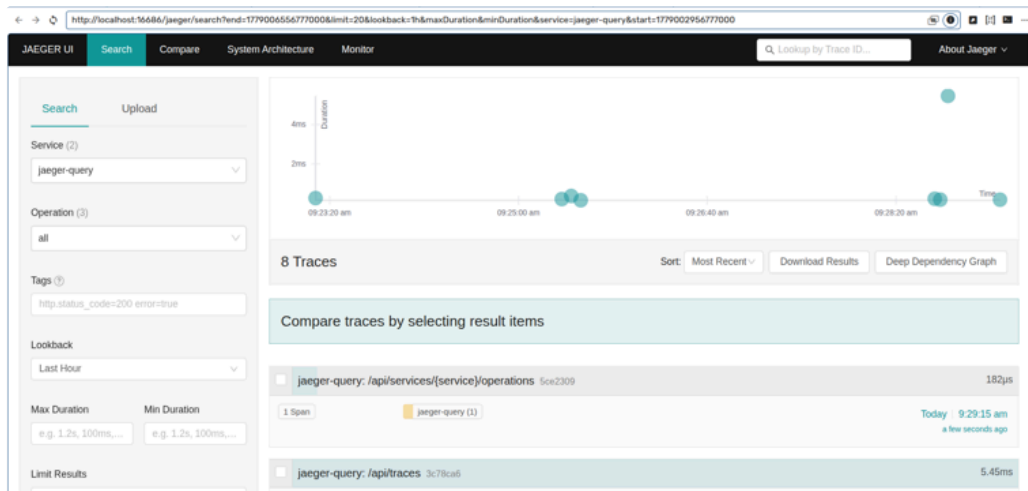
6.2 Distributed Tracing - Jaeger

The Jaeger is capturing 100% of traces, which is thanks to the `Telemetry` resource in the `svcp` namespace. The screenshot contains captures of 1500 traces from the last hour from the service `istio-ingressgateway.istio-system`. These traces show all five operations, wherein there are requests to `/api/customers` and `/api/users` on the customer-user service, as well as requests to the planning service. All traces display the full route from the ingress to the backend service. The `/api/users` 401ms trace stands out from sub-10ms responses over the same period. This was likely due to either a cold start or initialisation from the underlying database on that request.





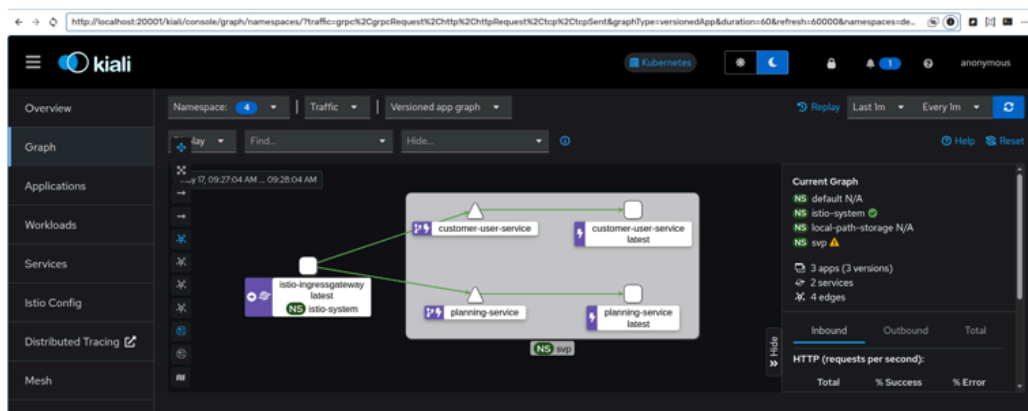
Gateway after Functional Tests:



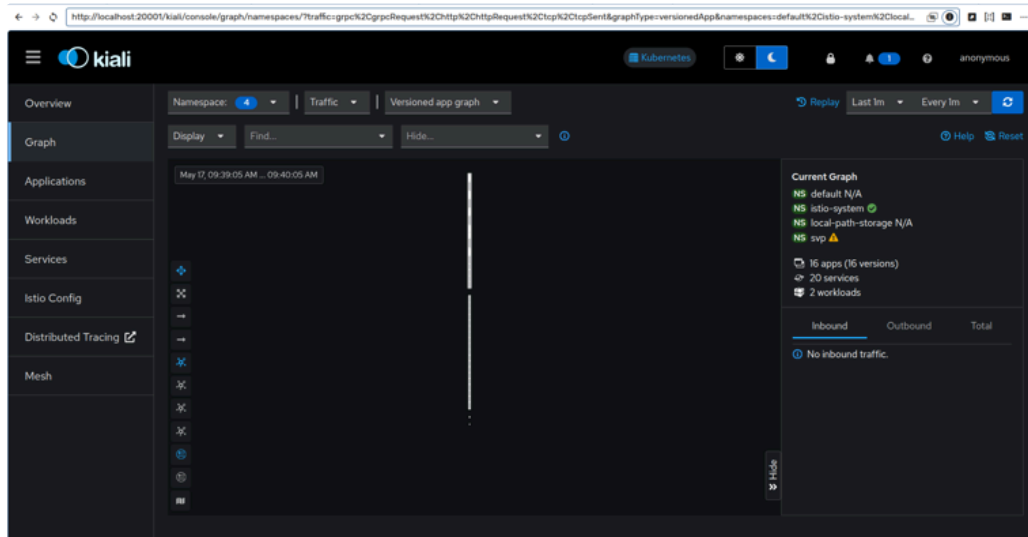
6.3 Topology Visualisation - Kiali

The service graph for the `svp` namespace is displayed by Kiali.

The graph shown in image 1 was taken before the tests ran and there was no inbound traffic.



The graph depicted in Image 2 illustrates the ingress gateway routing traffic to both the customer-user service and the planning service after the functional tests in service, while the connections between service and workload nodes are clear. The `svp` namespace is flagged with a warning icon, which in this example indicates the namespace health check, not anything wrong with the service.



7. Testing

The test suite is split into two separate runs: functional and stress.

The stress suite manages its own port-forwards and applies Istio fault injection manifests that would interfere with the functional suite endpoints. They cannot be ran together.

We start with the Functional tests followed by the Stress Tests.

```
1 | pytest tests/functional/ -v
2 | pytest tests/stress/ -v
```

When running the tests, make sure to have the port forwards up if the auto boot is failing.

```
1 | kubectl port-forward -n istio-system svc/istio-ingressgateway 8080:80 &
2 | kubectl port-forward -n istio-system svc/prometheus 9090:9090 &
3 | kubectl port-forward -n istio-system svc/tracing 16686:80 &
```

7.1 Functional Tests

Tests from the functional suite are spread across 4 subdirectories, e2e, integration, regression and unit. 379 tests in total. The integration testing assesses the performance of Kafka saga flows and resources such as Redis and CQRS in our system. The helm templates, istio manifest YAML and network policy YAML were validated with unit tests without needing a running cluster. All 379 tests are successful.

```
(.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ pytest tests/functional/ --tb=no -q
..... [ 35%]
..... [ 71%]
..... [100%]
[log] test output written to /home/duck/Documents/college/year4/microservices/project/svp_mic/test_logs/functional_all_1030_20260517.log
379 passed in 209.33s (0:03:29)
```

7.2 Stress Tests

The suite contains a total of 89 tests grouped into concurrency and resilience subfolders. The tests executed concurrently to jointly test bursts and measurements across all service endpoints. The resilience tests involve injecting faults, failing pods, and rate limiting. Out of 89, 88 pass consistently.

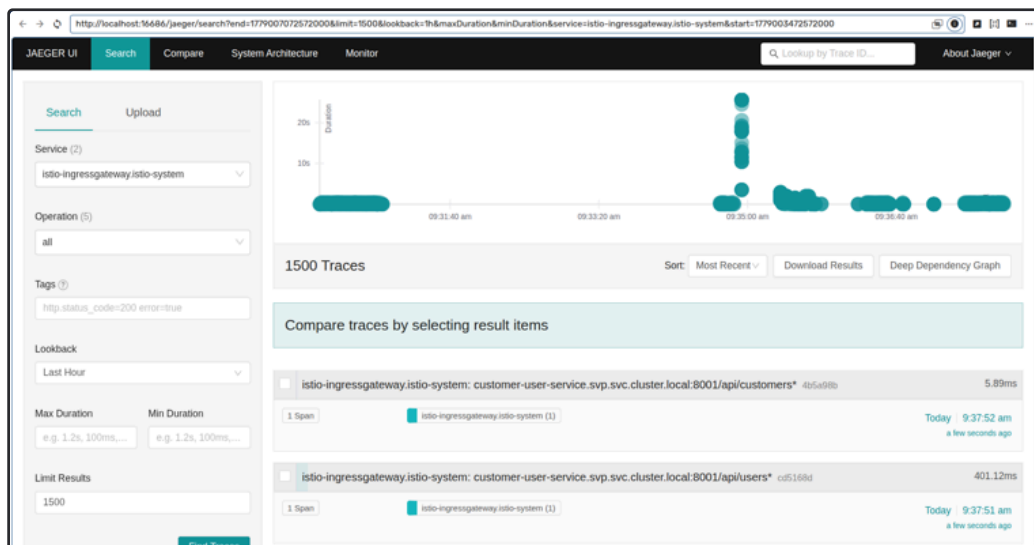
This is a failure in the fault injection suite. The user service is set at an arbitrary 50% fault rate; there will be 20 samples to test.

With just 20 samples and a 50% falsity rate the outcome has high variance, so the test sometimes tips the wrong way. The design of the test is a statistical problem, the system is not defective.

```
(.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ pytest tests/stress/ --tb=no -q
..... [100%]
[log] test output written to /home/duck/Documents/college/year4/microservices/project/svp_mic/test_logs/stress_all_1046_20260517.log
89 passed in 935.61s (0:15:35)
```

7.3 Fault Injection

For each test case, two fault injection manifests are applied and removed from `tests/stress/resilience/manifests/`. The delay manifest applies a latency of 3 seconds to half of the incoming customer-user requests. The expense service abort manifest will return an HTTP 503 on every request. Since both are executed in the VirtualService layer, the application code is unaware of them.



7.4 Pod Failure

The Kubernetes API is used to directly delete pods in the failure tests and verify service recovery. Once the replacement pod is ready, Kubernetes reschedules the pod that was deleted and the endpoint returns healthy responses. Once the stress suite tests are done running, the teardown restarts all deployments to make the cluster clean.

```
(.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ pytest tests/stress/ --tb=no -q
[log] test output written to /home/duck/Documents/college/year4/microservices/project/svp_mic/test_logs/stress_all_1046_20260517.log [100%]
89 passed in 935.61s (0:15:35)
(.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$
(.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$ kubectl get pods -n svp
NAME                                READY   STATUS    RESTARTS   AGE
customer-user-db-0                  1/1     Running   0           52m
expense-db-0                        1/1     Running   0           52m
kafka-0                            1/1     Running   2 (23m ago) 23m
planning-db-0                      1/1     Running   0           52m
redis-0                            2/2     Running   0           20m
svp-customer-user-service-7dd847c584-dgshh 2/2     Running   0           19m
svp-expense-service-74b54cdd5d-bnpzb 2/2     Running   0           19m
svp-frontend-service-6665f9c7f6-bh9ft 2/2     Running   0           19m
svp-planning-service-694546fdf8-fl7m9 2/2     Running   0           19m
zookeeper-0                        1/1     Running   0           52m
(.venv) duck@duckpad:~/Documents/college/year4/microservices/project/svp_mic$
```

7.5 Rate Limiting

The customer-user service has a local token bucket of 10 requests/sec via the EnvoyFilter. HTTP 429 rocks up with the `x-local-rate-limit: true` header response. The rate limiting tests check this behaviour by sending bursts over the rate limit threshold and asserting 429 responses are returned.

8. Reflection

Overall I found this stage of the project more enjoyable. If Stage 1 was what made Microservices as a whole click, Stage 2 showed me the power it can have (as well as the headaches that come with it).

Once again, the biggest shame remains that I did not learn this before my Internship in IFM and the 4th Year Project.

A positive is that i used this project to figure out how Calico works with Istio and how to test with them.

The application has some issues, mainly the lack of Authentication and Authorisation, but that was not in the scope of either stage.

9. Use of AI

AI was used throughout the code aspect of this project as per the project spec.

All code was generated using Claude Code CLI.

All prompts were written by myself, Luke Doyle.

To keep prompts consistent across the project, I defined a template at the start and used it for every prompt from scaffolding through to the final service. Rather than restating file paths, import structures, and code standards in every prompt, I described what I wanted in plain terms and used AI to convert that into a properly formatted prompt following the template. This meant every prompt going into Claude Code was structured the same way and I was not repeating myself.

Platform Version of Template:

```
1 [ACTION] [what is being built]
2
3 context:
4 [relevant background the agent needs before touching any files]
5
6 files to create/change:
7 - [path]
8 - [path]
9
10 [filename]:
11 [detailed spec for this file]
12   [sub-spec]
13   [sub-spec]
14
15 [filename]:
16 [detailed spec]
17
18 imports / dependencies:
19 [what this depends on, what it imports from where]
20
21 standards:
22 follow kubernetes best practices.
23 use resource requests and limits on all containers.
24 add liveness and readiness probes where appropriate.
```

For more details on the prompts used, go to `./docs/prompts` in the project directory.